```python
from __future__ import annotations

import json
import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    average_precision_score,
    confusion_matrix,
    precision_recall_curve,
    precision_recall_fscore_support,
    roc_auc_score,
    roc_curve,
)


PROJECT_ROOT = Path(__file__).resolve().parents[1]
INPUT_PATH = PROJECT_ROOT / "data" / "scored_access_logs.csv"
REPORTS_DIR = PROJECT_ROOT / "reports"

METRICS_PATH = REPORTS_DIR / "evaluation_metrics.json"
SUMMARY_PATH = REPORTS_DIR / "model_evaluation_summary.md"


def save_binary_confusion_matrix(
    actual: pd.Series,
    predicted: pd.Series,
) -> None:
    matrix = confusion_matrix(actual, predicted, labels=[0, 1])

    display = ConfusionMatrixDisplay(
        confusion_matrix=matrix,
        display_labels=["Normal", "Anomaly"],
    )
    display.plot(cmap="Blues", values_format="d")
    plt.title("Binary Anomaly Detection Confusion Matrix")
    plt.tight_layout()
    plt.savefig(
        REPORTS_DIR / "binary_confusion_matrix.png",
        dpi=180,
        bbox_inches="tight",
    )
    plt.close()


def save_roc_curve(actual: pd.Series, scores: pd.Series) -> None:
    false_positive_rate, true_positive_rate, _ = roc_curve(actual, scores)
    auc_score = roc_auc_score(actual, scores)

    plt.figure(figsize=(7, 5))
    plt.plot(
        false_positive_rate,
        true_positive_rate,
        label=f"Hybrid model (AUC = {auc_score:.4f})",
        linewidth=2,
    )
    plt.plot([0, 1], [0, 1], linestyle="--", color="grey")
    plt.xlabel("False positive rate")
    plt.ylabel("True positive rate")
    plt.title("ROC Curve: Anomaly Detection")
    plt.legend(loc="lower right")
    plt.grid(alpha=0.25)
    plt.tight_layout()
    plt.savefig(
        REPORTS_DIR / "roc_curve.png",
        dpi=180,
        bbox_inches="tight",
    )
    plt.close()


def save_precision_recall_curve(
    actual: pd.Series,
    scores: pd.Series,
) -> None:
    precision, recall, _ = precision_recall_curve(actual, scores)
    average_precision = average_precision_score(actual, scores)

    plt.figure(figsize=(7, 5))
    plt.plot(
        recall,
        precision,
        label=f"Hybrid model (AP = {average_precision:.4f})",
        linewidth=2,
    )
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("PrecisionвЂ“Recall Curve: Imbalanced Anomaly Data")
    plt.legend(loc="lower left")
    plt.grid(alpha=0.25)
    plt.tight_layout()
    plt.savefig(
        REPORTS_DIR / "precision_recall_curve.png",
        dpi=180,
        bbox_inches="tight",
    )
    plt.close()


def save_attack_type_confusion_matrix(anomalies: pd.DataFrame) -> None:
    labels = sorted(
        set(anomalies["anomaly_type"])
        | set(anomalies["predicted_attack_type"])
    )

    matrix = confusion_matrix(
        anomalies["anomaly_type"],
        anomalies["predicted_attack_type"],
        labels=labels,
    )

    figure, axis = plt.subplots(
        figsize=(max(8, len(labels) * 1.45), max(6, len(labels) * 1.25))
    )
    image = axis.imshow(matrix, cmap="Purples")

    axis.set_xticks(np.arange(len(labels)))
    axis.set_yticks(np.arange(len(labels)))
```

```
    axis.set_xticklabels(
        [label.replace("_", "\n") for label in labels],
        rotation=30,
        ha="right",
    )
    axis.set_yticklabels([label.replace("_", "\n") for label in labels])

    axis.set_xlabel("Predicted attack type")
    axis.set_ylabel("Actual attack type")
    axis.set_title("Attack-Type Classification Confusion Matrix")

    threshold = matrix.max() / 2 if matrix.max() else 0

    for row in range(matrix.shape[0]):
        for column in range(matrix.shape[1]):
            axis.text(
                column,
                row,
                str(matrix[row, column]),
                ha="center",
                va="center",
                color="white" if matrix[row, column] > threshold else "black",
            )

    figure.colorbar(image, ax=axis, label="Events")
    plt.tight_layout()
    plt.savefig(
        REPORTS_DIR / "attack_type_confusion_matrix.png",
        dpi=180,
        bbox_inches="tight",
    )
    plt.close()


def save_risk_by_label(data: pd.DataFrame) -> None:
    plot_data = data.copy()
    plot_data["Actual class"] = np.where(
        plot_data["is_anomaly"] == 1,
        "Injected anomaly",
        "Normal / benign edge case",
    )

    classes = ["Normal / benign edge case", "Injected anomaly"]
    grouped_scores = [
        plot_data.loc[
            plot_data["Actual class"] == class_name,
            "risk_score",
        ].values
        for class_name in classes
    ]

    plt.figure(figsize=(8, 5))
    box = plt.boxplot(grouped_scores, tick_labels=classes, patch_artist=True)

    for patch, color in zip(box["boxes"], ["#8ecae6", "#e76f51"]):
        patch.set_facecolor(color)

    plt.ylabel("Risk score")
    plt.title("Risk-Score Separation Between Normal and Anomalous Events")
    plt.grid(axis="y", alpha=0.25)
    plt.tight_layout()
    plt.savefig(
        REPORTS_DIR / "risk_score_separation.png",
        dpi=180,
        bbox_inches="tight",
    )
    plt.close()


def write_markdown_summary(metrics: dict) -> None:
    report = f"""# SentinelAI model evaluation summary

## Dataset and assumptions

- **Hold-out future access events:** {metrics["total_events"]:,}
- **Injected anomaly events:** {metrics["total_anomalies"]:,}
- **Anomaly rate:** {metrics["anomaly_rate"]:.2%}
- **Normal / benign events:** {metrics["total_normal_events"]:,}
- The first 14 days are used as each entity's initial behavioural baseline.
- Metrics use only the final chronological 25% of events, never seen during model fitting.

## Binary anomaly-detection performance

| Metric | Result |
|---|---:|
| Precision | {metrics["precision"]:.4f} |
| Recall | {metrics["recall"]:.4f} |
| F1 score | {metrics["f1_score"]:.4f} |
| ROC-AUC | {metrics["roc_auc"]:.4f} |
| Average precision | {metrics["average_precision"]:.4f} |
| False-positive rate | {metrics["false_positive_rate"]:.4%} |

## Analyst alert-budget performance

At the top 1% of events ({metrics["top_one_percent_alerts"]} alerts), the system achieves:

- **Precision at top 1%:** {metrics["precision_at_top_one_percent"]:.2%}
- **Recall at top 1%:** {metrics["recall_at_top_one_percent"]:.2%}

This reflects a realistic SOC-style alert budget, where analysts review only the highest-risk events first.

## Anomaly-type classification

- **Attack-type classification accuracy:** {metrics["attack_type_accuracy"]:.2%}
- Attack categories: brute force, credential stuffing, impossible travel, lateral movement, device spoofing, and low-and-slow exfiltration.

## Cold start and concept drift

- **Cold-start events:** {metrics["cold_start_events"]:,}
- **Cold-start alert rate:** {metrics["cold_start_alert_rate"]:.2%}
- **Benign insider-drift events:** {metrics["insider_drift_events"]:,}
- **Insider-drift false-positive rate:** {metrics["insider_drift_alert_rate"]:.2%}

The benign insider-drift behaviour is deliberately included so that the system does not assume every new resource is a confirmed intrusion.

## Explainability

Each alert provides a 0–100 risk score, a predicted attack type, and concrete contributing factors such as:

- rapid failed logins from one source IP;
- one source targeting many identities;
- new device fingerprint or geographic location;
- impossible travel;
```

- unusual resource or command sequence;
- off-hours access and unusually large transfers.

Limitations and next steps

1. The dataset is synthetic; real deployment requires validation against production logs and privacy controls.
2. Attack patterns are controlled and therefore cleaner than real adversarial behaviour.
3. The project uses safe repeated observations to adapt profiles; production still needs analyst feedback and drift monitoring.
4. A production system should add secure model serving, authentication, audit logging, and alert integrations.
"""

    SUMMARY_PATH.write_text(report, encoding="utf-8")

def main() -> None:
    if not INPUT_PATH.exists():
        raise FileNotFoundError(
            "Scored data is missing. Run: python src\\detector.py"
)

    REPORTS_DIR.mkdir(exist_ok=True)

    data = pd.read_csv(INPUT_PATH)
    # Do not mix training events into the reported evaluation metrics.
    if "evaluation_partition" in data.columns:
        data = data[data["evaluation_partition"] == "test"].copy()
    actual = data["is_anomaly"]
    predicted = data["predicted_anomaly"]
    scores = data["risk_score"] / 100

    precision, recall, f1_score, _ = precision_recall_fscore_support(
        actual,
        predicted,
        average="binary",
        zero_division=0,
)

    matrix = confusion_matrix(actual, predicted, labels=[0, 1])
    true_negative, false_positive, false_negative, true_positive = matrix.ravel()

    top_count = max(1, math.ceil(len(data) * 0.01))
    top_alerts = data.nlargest(top_count, "risk_score")

    anomalies = data[data["is_anomaly"] == 1].copy()
    attack_type_accuracy = accuracy_score(
        anomalies["anomaly_type"],
        anomalies["predicted_attack_type"],
)

    cold_start = data[data["cold_start_entity"] == 1]
    insider_drift = data[data["anomaly_type"] == "insider_drift"]

    metrics = {
        "total_events": int(len(data)),
        "total_anomalies": int(actual.sum()),
        "total_normal_events": int((actual == 0).sum()),
        "anomaly_rate": float(actual.mean()),
        "true_negative": int(true_negative),
        "false_positive": int(false_positive),
        "false_negative": int(false_negative),
        "true_positive": int(true_positive),
        "precision": float(precision),
        "recall": float(recall),
        "f1_score": float(f1_score),
        "roc_auc": float(roc_auc_score(actual, scores)),
        "average_precision": float(average_precision_score(actual, scores)),
        "false_positive_rate": float(
            false_positive / (false_positive + true_negative)
        ),
        "top_one_percent_alerts": int(top_count),
        "precision_at_top_one_percent": float(
            top_alerts["is_anomaly"].mean()
        ),
        "recall_at_top_one_percent": float(
            top_alerts["is_anomaly"].sum() / actual.sum()
        ),
        "attack_type_accuracy": float(attack_type_accuracy),
        "cold_start_events": int(len(cold_start)),
        "cold_start_alert_rate": float(
            cold_start["predicted_anomaly"].mean()
)
        if len(cold_start)
        else 0.0,
        "insider_drift_events": int(len(insider_drift)),
        "insider_drift_alert_rate": float(
            insider_drift["predicted_anomaly"].mean()
)
        if len(insider_drift)
        else 0.0,
    }

    save_binary_confusion_matrix(actual, predicted)
    save_roc_curve(actual, scores)
    save_precision_recall_curve(actual, scores)
    save_attack_type_confusion_matrix(anomalies)
    save_risk_by_label(data)

    with open(METRICS_PATH, "w", encoding="utf-8") as file:
        json.dump(metrics, file, indent=2)

    write_markdown_summary(metrics)

    print("Evaluation complete.")
    print(f"Metrics: {METRICS_PATH}")
    print(f"Written summary: {SUMMARY_PATH}")
    print("\nKey results:")
    print(f"Precision: {metrics['precision']:.4f}")
    print(f"Recall: {metrics['recall']:.4f}")
    print(f"F1 score: {metrics['f1_score']:.4f}")
    print(f"ROC-AUC: {metrics['roc_auc']:.4f}")
    print(
        "Attack-type accuracy: "
        f"{metrics['attack_type_accuracy']:.2%}"
)
    print(
        "Insider-drift false-positive rate: "
        f"{metrics['insider_drift_alert_rate']:.2%}"
)

if __name__ == "__main__":
    main()